

Analizador léxico y semántico PF2025

Lenguajes y Autómatas II | Informe técnico | 21 de septiembre de 2026

Índice

1. Introducción y objetivo	1
2. Desarrollo: formato y gramática de sentencias	2
3. Gramática de expresiones y reglas de tipos	3
4. Implementación: fases, símbolos y pila	4
5. Esquemas de traducción y acciones semánticas	5
6. Árboles anotados	6
7. Gestión de errores y archivos de salida	7
8. Pruebas y correspondencia con la rúbrica	8
9. Conclusiones, alcance y referencias	9

1. Introducción y objetivo

El análisis léxico transforma el texto de un programa en tokens. El análisis sintáctico verifica cómo se combinan esos tokens. El análisis semántico comprueba que las operaciones tengan sentido según las declaraciones y los tipos del lenguaje. Por ejemplo, una suma entre un entero y un booleano puede ser una expresión reconocible, pero debe rechazarse por incompatibilidad de tipos.

Este proyecto integra las tres tareas en una aplicación de consola escrita en Python 3. El objetivo es analizar el archivo `entrada/progfte.txt`, detectar errores con posiciones originales y producir evidencia verificable: tabla de símbolos, tokens, diagnósticos y un árbol de sintaxis abstracta (AST) anotado con tipos.

La base de evaluación es la rúbrica U1 de análisis semántico proporcionada por el docente Marco Antonio Guzmán S. [1]. Por indicación del usuario, se excluyen únicamente las comprobaciones que exigen un argumento entero en `printInt` y booleano en `printBool`. En PF2025 sus equivalentes son `impdig` e `impBool`. Sus argumentos siguen analizándose para detectar variables inexistentes u operaciones incompatibles.

El informe se entrega sin portada por solicitud del usuario. Incluye las secciones técnicas de la rúbrica y una propuesta de reflexión personal para que cada integrante la adapte. No acredita la explicación individual ni la fecha de entrega en plataforma.

Objetivos específicos

- Verificar declaraciones, duplicados, expresiones, asignaciones, condiciones y argumentos.
- Construir un AST real y utilizar una pila semántica para reducir expresiones.
- Conservar renglón, columna, fase, clase de error y tipos implicados.
- Mantener el programa de referencia intacto y añadir pruebas reproducibles.

2. Desarrollo: formato y gramática de sentencias

PF2025 distingue mayúsculas y minúsculas. Los tipos se escriben `Ent`, `cad` y `Bool`; sus nombres internos son `ent`, `cad` y `bool`. Los booleanos literales son `verdadero` y `falso`. Todas las variables se declaran en la sección global anterior a `inicio`. No existen declaraciones locales ni sombreado de nombres.

Un identificador comienza con letra y continúa con letras o dígitos. La implementación admite letras Unicode mediante los predicados de Python; rechaza `_`. Los enteros se reconocen como secuencias de dígitos; el signo negativo es un operador unario. Las cadenas ocupan una línea, se delimitan con comillas dobles y no implementan secuencias de escape. Los comentarios `/* ... */` pueden abarcar varias líneas, pero no se anidan.

Gramática de sentencias

La notación `{ X }` indica repetición, `[ X ]` un elemento opcional y `|` una alternativa. `id`, `entero` y `cadena` son categorías léxicas.

```
programa      ::= pf2025 id [ decl declaraciones ]
                  inicio sentencias fin
declaraciones ::= { tipo id { , id } ; }
tipo          ::= Ent | cad | Bool
sentencias    ::= { sentencia }
sentencia     ::= asignacion | llamada | condicional | bucle
asignacion    ::= id := expresion ;
llamada       ::= leerdig ( id ) ; | leercad ( id ) ;
                  | impcad ( expresion ) ;
                  | impdig ( expresion ) ;
                  | impBool ( expresion ) ;
condicional   ::= si expresion entonces sentencias
                  [ sino sentencias ] fin si [ ; ]
bucle         ::= mientras expresion hacer
                  sentencias finmientras [ ; ]
```

Ejemplo válido con ciclo

```
pf2025 Cuenta
decl Ent contador;
inicio
    contador := 3;
    mientras contador > 0 hacer
        impdig(contador);
        contador := contador - 1;
    finmientras;
fin
```

El ejemplo se analiza estáticamente: no imprime ni ejecuta el ciclo. Las palabras de control están en español, de acuerdo con la nota de la rúbrica.

3. Gramática de expresiones y reglas de tipos

La precedencia aumenta desde o hasta los primarios. Los operadores binarios aritméticos se asocian a la izquierda. Una comparación no puede encadenarse directamente: se escribe $a < b$ y $b < c$, no $a < b < c$. Los paréntesis permiten agrupar expresiones.

```
expresion      ::= disyuncion
disyuncion     ::= conjuncion { o conjuncion }
conjuncion     ::= negacion { y negacion }
negacion       ::= no negacion | relacional
relacional     ::= adicion [ oprel adicion ]
oprel          ::= == | != | > | < | >= | <=
adicion        ::= producto { ( + | - ) producto }
producto       ::= unario { ( * | / ) unario }
unario         ::= - unario | primario
primario       ::= id | entero | cadena | verdadero | falso
               | ( expresion )
```

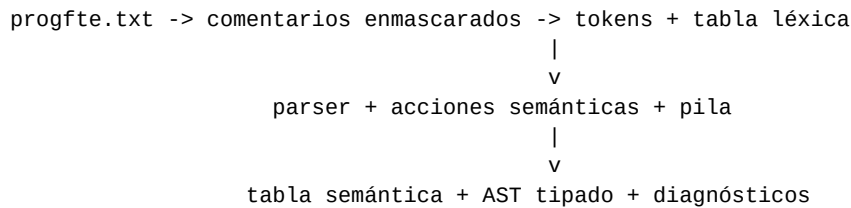
La implementación usa precedencias numéricas: o=1, y=2, no=3, relaciones =4, suma/resta =5, producto/división =6 y negación aritmética =7. Los prefijos se reconocen al comenzar una subexpresión; una combinación incompatible se rechaza por sus tipos.

Construcción	Condición	Resultado
Entero / cadena / booleano	Literal reconocido	ent / cad / bool
Identificador	Declarado en ámbito global	Tipo declarado
$E1 + E2$, $E1 - E2$, $E1 * E2$, $E1 / E2$	Ambos ent	ent
$-E$	E es ent	ent
$E1 == E2$, $E1 != E2$	Mismo tipo	bool
$E1 > E2$, $E1 < E2$, $E1 >= E2$, $E1 <= E2$	Ambos ent	bool
$E1$ y $E2$, $E1$ o $E2$, no E	Operandos bool	bool
$id := E$	Tipo de id igual al de E	Sentencia válida
si E / mientras E	E es bool; revisar el cuerpo	Sentencia válida
leerdig(id) / leercad(id)	Variable ent / cad	Lectura válida
impcad(E)	E es cad	Impresión válida
impdig(E) / impBool(E)	Analizar E, sin imponer tipo final	Excepción solicitada

Un tipo None representa una expresión ya inválida. Se propaga para evitar errores derivados redundantes. La división produce tipo ent en este análisis; no se calculan valores ni se comprueba la división por cero en ejecución.

4. Implementación: fases, símbolos y pila

Flujo integrado



`AnalizadorLexico` conserva dos representaciones. `lineas_lexicas` sustituye los caracteres de los comentarios por espacios para mantener las coordenadas; `codigo_depurado` sirve para generar `.dep`. El tokenizador utiliza la primera. Los comentarios dentro de cadenas son texto ordinario y un comentario abierto al final del archivo produce un error léxico.

`AnalizadorSemantico` consume la lista de tokens con un cursor. Durante ese recorrido reconoce sentencias, consulta declaraciones, reduce expresiones y construye el AST. No recorre el AST nuevamente para calcular tipos. El ordenamiento final de diagnósticos y la serialización son tareas de presentación posteriores.

Tablas de símbolos y ámbito

La tabla léxica conserva cada lexema distinto con su token y referencia; un diccionario permite reutilizar referencias de lexemas repetidos. La tabla semántica registra por nombre el tipo, ámbito global y coordenadas de declaración. La consulta `_tipo_variable` utiliza esta tabla; una redeclaración genera un error y conserva la primera declaración.

Las declaraciones con un error léxico se omiten para evitar incorporar símbolos parciales. En la entrada de referencia, `nombre@Usuario` es inválido y no declara `nombreUsuario`. Las variables globales son visibles dentro de los condicionales y ciclos. Una declaración en el cuerpo se rechaza como sintaxis inválida.

Pila semántica y AST

Cada primario apila un nodo con clase, lexema, posición y tipo. `_reducir` extrae uno o dos nodos, comprueba sus tipos y apila el nodo del operador con sus hijos. `_expresion` extrae el resultado para incorporarlo a la sentencia y verifica que la pila recupere su tamaño inicial. Después de analizar una sentencia completa no quedan operandos pendientes.

El AST contiene un nodo `Programa`, declaraciones y cuerpo; las sentencias contienen sus expresiones, condiciones y bloques hijos. Los nodos inválidos conservan tipo nulo. El archivo `.ast.json` permite inspeccionar la jerarquía y los atributos, sin depender de dibujos manuales. Su serialización usa `json` de la biblioteca estándar [3].

5. Esquemas de traducción y acciones semánticas

Se usa T como tabla semántica, P como pila y n.tipo como atributo de un nodo. error registra fase, clase, posición, tipos y descripción. Cada acción ocurre al reconocer su producción; no se requiere una segunda evaluación del árbol.

Producción o evento	Acción implementada
tipo id	Si id pertenece a T, informar duplicado; en otro caso insertar tipo, ámbito y posición. Crear nodo Declaracion.
id como expresión	Consultar T; si no existe, informar variable no declarada. Apilar nodo Variable con su tipo o tipo nulo.
literal	Apilar nodo Literal con ent, cad o bool.
(E)	Consumir delimitadores y sustituir el nodo superior por Grupo, conservando el tipo de E.
operador unario E	Extraer E; comprobar tipo; apilar Unario con hijo E y tipo resultante.
E1 operador E2	Extraer E2 y E1; comprobar reglas de tipos; apilar Binario con hijos ordenados.
id := E ;	Consultar id, consumir resultado de E, comparar tipos y construir Asignacion.
lectura (E) ;	Consumir E; exigir un nodo Variable y el tipo requerido por la lectura; construir Llamada.
impresión (E) ;	Consumir E; exigir cad solamente para impcad; construir Llamada.
si E entonces B1 sino B2 fin si	Exigir bool en E; analizar ambos bloques; construir Si con condición, cuerpo y alternativa.
mientras E hacer B fin mientras	Exigir bool en E; analizar B una vez; construir Mientras con condición y cuerpo.
programa completo	Reunir declaraciones y sentencias en Programa; exportar árbol y diagnósticos.

Reducción de una suma

Al reconocer $E1 + E2$:

```
derecha = P.pop()
izquierda = P.pop()
si alguno tiene tipo nulo: resultado = nulo
en otro caso, si ambos son ent: resultado = ent
en otro caso: informar incompatibilidad; resultado = nulo
P.push(Binario('+', izquierda, derecha, resultado))
```

Para $1 + 2 * 3$, la pila evoluciona como $[1]$, $[1, 2]$, $[1, 2, 3]$, $[1, *(2, 3)]$ y $[(1, *(2, 3))]$. La sentencia extrae el último nodo y deja la pila vacía. La prioridad del producto se conserva en la estructura y no requiere ejecutar la operación.

6. Árboles anotados

Los ejemplos siguientes representan los nodos que construye el analizador. Las hojas muestran tipos declarados o literales; los operadores muestran el tipo sintetizado. Un tipo nulo indica que el resultado no puede considerarse válido.

Asignación y precedencia

Para `a := 1 + 2 * 3;`, con `a: ent`:

```
Asignacion a [ent]
  expresion: + [ent]
    izquierda: 1 [ent]
    derecha: * [ent]
      izquierda: 2 [ent]
      derecha: 3 [ent]
```

La multiplicación se agrupa antes que la suma. La asignación compara el tipo `ent` de su destino con el `ent` producido por la raíz de la expresión.

Condición y cuerpo de un ciclo

Para `mientras a > 0 hacer a := a - 1; finmientras;`:

```
Mientras
  condicion: > [bool]
    izquierda: a [ent]
    derecha: 0 [ent]
  cuerpo:
    Asignacion a [ent]
      expresion: - [ent]
        izquierda: a [ent]
        derecha: 1 [ent]
```

Se revisa tanto la condición como la asignación del cuerpo. El análisis no repite el cuerpo según el valor de `a`; solo lo visita como estructura del programa.

Expresión incompatible

Para `a := 1 + verdadero;`, con `a: ent`:

```
Asignacion a [ent]
  expresion: + [nulo; error semantico]
    izquierda: 1 [ent]
    derecha: verdadero [bool]
```

Se informa `Operandos incompatibles` en la posición de `+`, con tipos esperados `ent/ent` y obtenidos `ent/bool`. El tipo nulo se propaga y evita informar además una incompatibilidad artificial en la asignación.

7. Gestión de errores y archivos de salida

Cada diagnóstico tiene renglón y columna desde 1, fase, clase de error, tipos implicados y descripción. Las columnas corresponden a caracteres del texto original: un tabulador cuenta como un carácter, no como varias columnas visuales. Los errores se ordenan por renglón y columna, manteniendo varios diagnósticos si son necesarios en una misma línea.

Las fases son Lexico, Sintactico y Semantico. Un carácter inválido es léxico; la falta de un delimitador es sintáctica; una incompatibilidad de tipos es semántica. Se conserva esta distinción para no atribuir a los tipos un problema de estructura. En un error léxico, el tipo de dato se indica como no aplica; una variable sin declaración tiene tipo desconocido.

Ejemplo del formato

```
Renglon: 5, Columna: 1, Semantico,  
Tipo de error: Asignacion incompatible,  
Tipo de dato: esperado: ent; obtenido: bool,  
Se esperaba ent, se obtuvo bool
```

El ejemplo se divide visualmente para facilitar su lectura. En .tok, .sem y consola, cada diagnóstico se escribe en una sola línea. Los diccionarios internos también conservan tipo_esperado y tipo_obtenido por separado.

Archivo	Evidencia generada
progfte.dep	Código sin comentarios y con separación normalizada
progfte.tab	Tabla de lexemas, tokens y referencias
progfte.tok	Tokens reconocidos y diagnósticos unificados
progfte.sem	Variables, tipos, ámbito, posición de declaración y errores
progfte.ast.json	Árbol completo con tipos y coordenadas

Resultado del programa de referencia

Sin modificar entrada/progfte.txt, el resultado es **273 tokens, 61 símbolos léxicos, 5 variables y 3 errores**. El renglón 5 contiene el identificador inválido nombre@Usuario. Los renglones 65 y 67 usan nombreUsuario, que no fue declarado. Las columnas respectivas son 5, 9 y 8.

La recuperación sintáctica utiliza delimitadores y cierres de bloque para continuar después de una estructura inválida. Se garantiza el avance del cursor en el cuerpo para evitar que un token inesperado detenga indefinidamente el análisis. La recuperación busca diagnósticos útiles, sin garantizar que una entrada arbitrariamente malformada produzca un único error.

8. Pruebas y correspondencia con la rúbrica

Se ejecutaron **19 pruebas automatizadas**, varias con subcasos, mediante `unittest` [2]. Todas finalizaron correctamente. Las pruebas trabajan en memoria o en carpetas temporales; no alteran el programa fijo. Se verificó además que el archivo de entrada no tiene diferencias respecto de Git.

```
python -B -m unittest discover -s tests -v
Ran 19 tests
OK
```

```
python src/analizador_lexico.py
Total de errores: 3
```

Grupo de pruebas	Resultado comprobado
Referencia y salidas	Conteos esperados; AST exportable; campos del diagnóstico
Declaraciones y asignaciones	Variables inexistentes, duplicados e incompatibilidades
Operadores y precedencia	Aritmética, relaciones, lógica, unarios y estructura del AST
Condicionales y ciclos	Condiciones bool, errores en ambos cuerpos y anidamiento
Funciones	Lecturas con variables y tipos correctos; rechazo de literales
Excepciones de impresión	<code>impdig</code> e <code>impBool</code> admiten cualquier tipo final válido
Léxico y posiciones	Cadenas intactas, comentarios abiertos, identificadores y <code>!=</code>
Recuperación	Expresiones incompletas, cierres ausentes, pila balanceada
Propagación	Un error de variable no causa incompatibilidades redundantes

Correspondencia con los aspectos evaluados

Funcionamiento (40%). Hay verificaciones de tipos, variables y ámbito global, análisis de expresiones aritméticas, argumentos, asignaciones y diagnósticos con coordenadas. Los tipos se calculan durante el recorrido de tokens. Evaluar expresiones se entiende aquí como determinar su validez y tipo, no ejecutar un intérprete.

Código (20%). La implementación está comentada, construye AST, alimenta y consulta tablas de símbolos y reduce nodos mediante una pila semántica. La explicación oral debe ser realizada por cada integrante.

Documentación (20%). Este informe incluye gramáticas, esquemas de traducción, árboles anotados, acciones semánticas, conclusiones y referencias. Se omite la portada por solicitud expresa.

Explicación (10%) y tiempo y forma (10%). No se acreditan mediante pruebas de software. La rúbrica indica como límite ordinario el 18 de septiembre de 2026; este informe no certifica cuándo se realizó la entrega. Las fases sí están conectadas en un único flujo de ejecución.

9. Conclusiones, alcance y referencias

Conclusiones técnicas

La corrección principal consiste en relacionar el análisis con estructuras verificables. Las declaraciones alimentan una tabla semántica; las expresiones producen nodos tipados; los operadores consumen operandos reales de la pila; las sentencias incorporan esos nodos al AST. Esto permite comprobar el comportamiento mediante pruebas y explicar el resultado a partir del árbol.

La separación entre texto para análisis y texto depurado evita perder la ubicación de los errores. Los ciclos se reconocen desde la fase léxica y sus condiciones y cuerpos se comprueban semánticamente. Los argumentos de lectura deben ser variables del tipo correspondiente. Las dos excepciones de impresión se mantienen sin desactivar la revisión interna de sus expresiones.

Las pruebas demuestran los casos incluidos y la conservación del resultado del programa de referencia. No constituyen una garantía para toda entrada posible. El lenguaje implementa un ámbito global, no ámbitos locales; analiza tipos, no valores en ejecución. El enunciado de la práctica 1 citado por la rúbrica no fue proporcionado y puede contener restricciones adicionales que deberán contrastarse si se dispone de él.

Propuesta de conclusión personal

Texto propuesto para que cada integrante adapte a su experiencia antes de entregar:

"Considero que la parte más importante de un analizador semántico es mantener la coherencia entre las declaraciones y el uso de las expresiones. El AST permite explicar la precedencia y la estructura del programa; la pila muestra cómo se obtiene el tipo de una operación a partir de sus operandos. También considero esencial que un mensaje indique dónde está el problema y cuáles son los tipos involucrados. Una mejora futura sería ampliar los casos de prueba y contrastar la gramática con todas las especificaciones originales de la práctica."

Referencias bibliográficas

[1] Guzmán S., Marco Antonio. *U1 Rúbrica Semántico Alumnos*. Instituto Tecnológico de Cd. Guzmán, Departamento de Sistemas y Computación, Lenguajes y Autómatas II. Documento de dos páginas proporcionado por el usuario; fecha límite indicada: 18 de septiembre de 2026.

[2] Python Software Foundation. *unittest - Unit testing framework*. Documentación oficial de Python. Consulta: 21 de septiembre de 2026. <https://docs.python.org/3.11/library/unittest.html>

[3] Python Software Foundation. *json - JSON encoder and decoder*. Documentación oficial de Python. Consulta: 21 de septiembre de 2026. <https://docs.python.org/3.13/library/json.html>

[4] Proyecto PF2025. *Código fuente y pruebas de aceptación*: `src/analizador_lexico.py` y `tests/test_rubrica.py`. Versión local revisada el 21 de septiembre de 2026. Evidencia primaria de los resultados de este informe.